

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 03 -

CONSTRUINDO UMA API DE ML COM FLASK

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	23
REFERÊNCIAS.....	24

LEANDRO

O QUE VEM POR AÍ?

Nesta aula vamos finalmente tirar o modelo do notebook e disponibilizá-lo com o Flask. Você vai ver, passo a passo, como criar uma aplicação web mínima: instanciar o app, definir rotas com `@app.route`, subir um “Olá, mundo” local e entender onde cada parte do código entra no fluxo HTTP. A partir daí, avançamos para o que realmente importa: carregar um modelo de ML já treinado, criar um endpoint `/predict` que recebe JSON, faz a inferência e devolve o resultado em formato estruturado.

Ao longo da aula você também verá como tratar erros básicos (entrada inválida, campos faltando), como testar a API e quais cuidados tomar para não transformar seu servidor local em uma gambiarra difícil de manter. A ideia é que, ao final, você tenha sua primeira API de ML em Flask rodando no localhost, entendendo exatamente o que está acontecendo em cada linha e pronto(a) para evoluir esse código posteriormente para algo mais robusto em produção.

HANDS ON

Prepare-se para colocar a mão na massa! Nesta aula, vamos além da teoria e partiremos para um Hands On completo, onde você aprenderá, passo a passo, a tirar um modelo de Machine Learning do ambiente do notebook e disponibilizar como uma API funcional usando Flask.

Veremos desde a criação de uma aplicação web mínima com rotas básicas, como o clássico "Olá, mundo", até a construção do endpoint `/predict` que carrega nosso modelo treinado, recebe dados via JSON, realiza a inferência e retorna o resultado estruturado, garantindo que ao final você tenha sua primeira API de ML rodando em localhost.

SAIBA MAIS

Introdução ao FLASK

Expor os modelos de ML como serviços web é essencial para gerar valor real com IA. Um modelo treinado parado em um jupyter notebook é apenas uma análise: para que ele tenha impacto, precisamos disponibilizá-lo via API para integração em aplicações do mundo real.

Como vimos nas aulas anteriores, uma API fornece uma interface padronizada para que softwares se comuniquem e troquem dados ou funcionalidades. Isso significa que, ao transformar o nosso modelo em um endpoint HTTP, ele pode enviar requisições e receber previsões em formato JSON. Isso permite a reutilização e integração do modelo em múltiplos sistemas de forma consistente.



Figura 1 – Flask
Fonte: Flask (2026)

O Flask é um microframework web em Python famoso por sua simplicidade e flexibilidade. O termo “microframework” significa que ele oferece apenas o essencial para construir aplicações web, sem impor uma estrutura pesada ou camadas complexas, deixando a pessoa desenvolvedora com total liberdade para escolher componentes adicionais conforme a necessidade do projeto.

Criado por Armin Ronacher em 2010, o Flask nasceu exatamente da necessidade de uma alternativa mais leve aos frameworks maiores (como o Django) para criar desde APIs RESTful simples até aplicações web completas.

Por sua abordagem minimalista, o Flask tornou-se popular para prototipação rápida e projetos de pequeno a médio porte. É como montar um Lego: você adiciona apenas o que precisa, mantendo o projeto enxuto. Isso traz vantagens práticas para APIs de Machine Learning pois, em poucos minutos, é possível ter um serviço de inferência rodando.

De fato, o Flask é muito indicado quando o objetivo é criar protótipos ágeis, serviços simples ou microsserviços especializados, evitando a sobrecarga de frameworks mais robustos. Em outras palavras, se a meta é expor rapidamente um modelo treinado para teste ou demonstração, o Flask costuma ser a primeira escolha.

Além disso, a curva de aprendizado é suave e, com poucas linhas de código, já conseguimos definir rotas e o que o serviço deve retornar. Essa combinação de facilidade e poder faz do Flask uma ferramenta valiosa para Data Scientists e engenheiros(as) que precisam "empacotar" modelos em APIs sem precisar virar especialistas em desenvolvimento web.

A Anatomia de uma API Básica com Flask

Vamos entender os componentes fundamentais de uma API Flask e como eles se encaixam.

Depois de instalarmos e importarmos o Flask, nós vamos criar uma instância da aplicação com `app = Flask(__name__)`. Essa instância será nossa aplicação web, responsável por registrar rotas e configurar o servidor. O parâmetro `__name__` é passado para ajudar o Flask a encontrar recursos relativos como templates ou arquivos estáticos, mas não vamos nos aprofundar nisso agora. O importante é que `app` será usado como um decorador para criar rotas e por fim para rodar a aplicação.

Definição de rotas (@app.route)

As rotas são os endpoints (URLs) da API. Com o decorador `@app.route("/caminho", methods=[...])` associamos uma função Python a um caminho URL e aos métodos HTTP suportados (GET, POST, etc.). Por exemplo, `@app.route("/hello", methods=["GET"])` poderia acionar uma função `hello()` que retorna um texto "Olá".

Ao receber uma requisição em `/hello`, o Flask invoca a função correspondente e envia sua resposta de volta. Podemos definir múltiplas rotas para diferentes funcionalidades e a ideia-chave é: cada rota mapeia uma URL para uma função seguindo o estilo REST, em que endpoints representam recursos ou ações.

Retorno em JSON

Em APIs de Machine Learning, é prática comum trabalhar com dados em formato JSON tanto na entrada quanto na saída, pela facilidade de integrar com diversos sistemas. O Flask facilita isso através da função utilitária `jsonify()`, que converte automaticamente dicionários (ou listas) Python para JSON e já configura o cabeçalho de resposta (Content-Type: application/json). Assim, em vez de retornar HTML como no exemplo anterior, podemos retornar objetos JSON. Exemplo: `return jsonify({"mensagem": "Olá, mundo!"})`. O Flask serializa o dicionário para `{"mensagem": "Olá, mundo!"}` e o cliente receberá esses dados estruturados.

O uso de JSON torna a API própria para consumo por outros sistemas: diferentemente de HTML (feito para humanos), o JSON é facilmente lido por aplicativos, linguagens diversas e até outros serviços. Com nossas rotas retornando JSON, abrimos caminho para que um app mobile, um frontend JavaScript ou outro microserviço consumam diretamente as previsões do modelo, de forma simples e estruturada.

Hands ON - Criando sua primeira API: “Olá Mundo”

VSCode

```
app_hello.py > ...
1
2 from flask import Flask
3
4 # Criando a aplicação Flask
5 app = Flask(__name__)
6
7 # Rota principal - GET por padrão
8 @app.route('/')
9 def home():
10     return 'Olá, Mundo! Bem-vindo à sua primeira API Flask!'
11
12 # Rota com parâmetro na URL
13 @app.route('/saudacao/<nome>')
14 def saudacao(nome):
15     return f'Olá, {nome}! Seja bem-vindo!'
16
17 # Executar a aplicação
18 if __name__ == '__main__':
19     app.run(port=8000)
20
```

Figura 2 – Hello Flask
Fonte: Elaborado pelo autor (2026)

Podemos rodar nosso código e testar no navegador acessando:

<http://127.0.0.1:8000/>

Para testar a saudação dos nomes, podemos colocar no caminho:

<http://127.0.0.1:8000/saudacao/loannis>

```
PS C:\B3-API> & C:/Users/Mastermind/AppData/Local/Python/pythoncore-3.14-64/python.exe c:/B3-API/app_hello.py
* Serving Flask app 'app_hello'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:8000
Press CTRL+C to quit
127.0.0.1 - - [05/Dec/2025 12:14:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 12:15:08] "GET /saudacao/Maria HTTP/1.1" 200 -
127.0.0.1 - - [05/Dec/2025 12:15:15] "GET /saudacao/Ioannis HTTP/1.1" 200 -
```

Figura 3 – Hello Flask - Terminal
Fonte: Elaborado pelo autor (2026)

Podemos testar também via CURL; nesse caso, pode ser tanto pelo terminal do VSCode como o CMD:

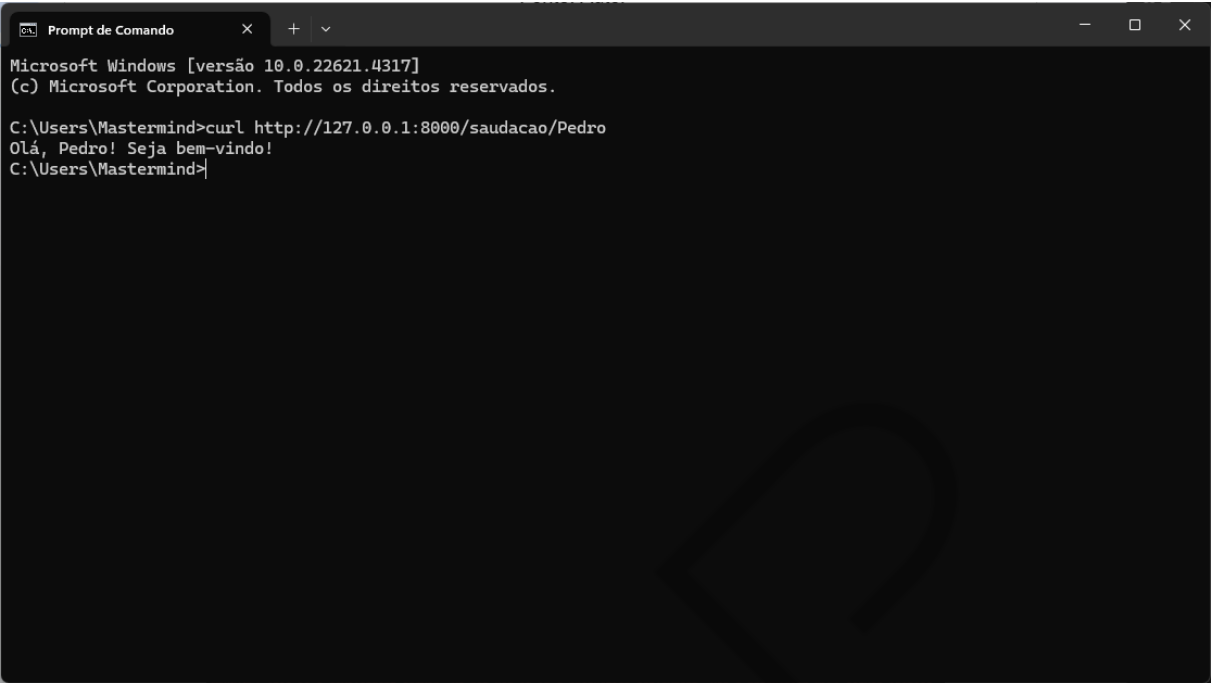


Figura 4 – Hello Flask - Curl
Fonte: Elaborado pelo autor (2026)

Trabalhando com JSON

JSON (JavaScript Object Notation) é o formato padrão para troca de dados em APIs REST, pois ele é leve e fácil de ler, suportado nativamente em praticamente todas as linguagens e tem estrutura de chave-valor, assim como nos dicionários Python.

O Flask oferece ferramentas para trabalhar com JSON:

FUNÇÃO	DESCRIÇÃO
<code>request.get_json()</code>	Extrai dados JSON do corpo da requisição
<code>jsonify()</code>	Converte dicionário Python para resposta JSON

Tabela 1 – Ferramentas do Flask para JSON
Fonte: Elaborado pelo autor (2026)

```
import json

# Dicionário Python
dados = {
    "nome": "João",
    "idade": 30,
    "cidade": "São Paulo",
    "skills": ["Python", "Machine Learning", "Flask"]
}

# Convertendo para JSON (similar ao que jsonify faz internamente)
json_string = json.dumps(dados, ensure_ascii=False, indent=2)
print("Dados em formato JSON:")
print(json_string)

# Parseando JSON de volta para dicionário (similar a request.get_json())
json_recebido = '{"valor": 42, "operacao": "multiplicar"}'
dados_parseados = json.loads(json_recebido)
print(f"\nJSON parseado: {dados_parseados}")
print(f"Tipo: {type(dados_parseados)}")
print(f"Acessando valor: {dados_parseados['valor']}")
```

✓ 0.0s Python

Figura 5 – jsonfy-vs
Fonte: Elaborado pelo autor (2026)

```
... Dados em formato JSON:
{
  "nome": "João",
  "idade": 30,
  "cidade": "São Paulo",
  "skills": [
    "Python",
    "Machine Learning",
    "Flask"
  ]
}

JSON parseado: {'valor': 42, 'operacao': 'multiplicar'}
Tipo: <class 'dict'>
Acessando valor: 42
```

Figura 6 – jsonfy-output
Fonte: Elaborado pelo autor (2026)

Agora temos a estrutura básica de uma API!

Integração com Modelo de ML

Chegamos ao coração da aula: colocar um modelo de Machine Learning para funcionar dentro de uma rota Flask. Suponha que você já tem um modelo treinado (por exemplo, um classificador de sklearn) salvo em disco – geralmente em formato pickle (.pkl) ou joblib (.joblib). A integração consiste nos seguintes passos: carregar o modelo, receber os dados de entrada da requisição, usar o modelo para prever e retornar o resultado ao cliente. Vamos por partes:

Treinando um Modelo de ML Simples

Antes de criar a API, precisamos de um modelo treinado. Vamos criar um modelo simples de classificação que preverá se uma flor é da espécie Iris Setosa, Versicolor ou Virginica. O dataset Iris é um dos datasets mais famosos em ML:

- 150 amostras de flores Iris.
- 4 features: comprimento/largura da sépala e pétala.
- 3 classes: Setosa, Versicolor, Virginica.



```
# Importando bibliotecas para ML
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pickle

# Carregando o dataset
iris = load_iris()
X = iris.data
y = iris.target

print(" Informações do Dataset:")
print(f"   Features: {iris.feature_names}")
print(f"   Classes: {list(iris.target_names)}")
print(f"   Total de amostras: {len(X)}")
print(f"   Shape dos dados: {X.shape}")
```

[1] ✓ 9.2s Python

... Informações do Dataset:
Features: ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Classes: [np.str_('setosa'), np.str_('versicolor'), np.str_('virginica')]
Total de amostras: 150
Shape dos dados: (150, 4)

Figura 7 – IRIS DATASET
Fonte: Elaborado pelo autor (2026)

Na sequência, dividimos o dataset em treino e teste:

```
# Dividindo em treino e teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"  Divisão dos dados:")
print(f"  Treino: {len(X_train)} amostras")
print(f"  Teste: {len(X_test)} amostras")
```

[3] ✓ 0.0s

... Divisão dos dados:
Treino: 120 amostras
Teste: 30 amostras

Figura 8 – IRIS TRAIN/TEST
Fonte: Elaborado pelo autor (2026)

Agora, treinamos o modelo com RandomForestClassifier e avaliamos os resultados:

```
# Treinando o modelo
modelo = RandomForestClassifier(n_estimators=100, random_state=42)
modelo.fit(X_train, y_train)

# Avaliando
y_pred = modelo.predict(X_test)
acuracia = accuracy_score(y_test, y_pred)

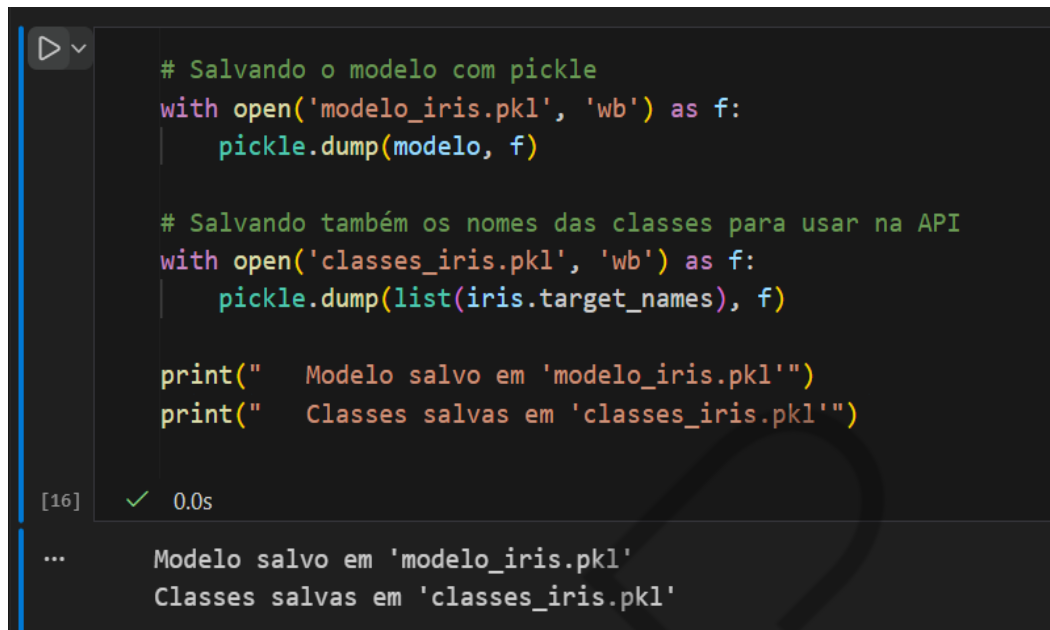
print(f"  Modelo treinado com sucesso!")
print(f"  Acurácia no teste: {acuracia:.2%}")
```

[12] ✓ 0.1s

... Modelo treinado com sucesso!
Acurácia no teste: 96.67%

Figura 9 – IRIS Model
Fonte: Elaborado pelo autor (2026)

E, por fim, vamos exportar nosso modelo com o formato pickle .pkl, para disponibilizar ele e os nomes das classes na nossa API:



```
# Salvando o modelo com pickle
with open('modelo_iris.pkl', 'wb') as f:
    pickle.dump(modelo, f)

# Salvando também os nomes das classes para usar na API
with open('classes_iris.pkl', 'wb') as f:
    pickle.dump(list(iris.target_names), f)

print("  Modelo salvo em 'modelo_iris.pkl'")
print("  Classes salvas em 'classes_iris.pkl'")
```

[16] ✓ 0.0s

... Modelo salvo em 'modelo_iris.pkl'
Classes salvas em 'classes_iris.pkl'

Figura 10 – IRIS Model
Fonte: Elaborado pelo autor (2026)

Construindo a API de ML

Agora que temos um modelo no nosso arquivo .pkl, vamos criar uma API completa que carregue esse modelo treinado e, através de uma rota POST, receba os dados JSON, faça a previsão conforme o modelo treinado e nos retorne o resultado também em JSON. É importante também adequar a API para tratamento de erros.

A estrutura da nossa API será essa:

MÉTODO	ENDPOINT	DESCRIÇÃO
GET	/	Informações sobre a API
GET	/health	Status de saúde da API
POST	/predict	Recebe features, retorna previsão

Tabela 2 – Estrutura da API
Fonte: Elaborado pelo autor (2026)

Tratamento de ERROS

Nenhuma API é completa sem considerar o que fazer quando algo dá errado. No contexto de nossa API Flask de ML, os erros mais prováveis são: (a) problemas na requisição do cliente (JSON malformatado, campos faltantes, tipos errados) ou (b) exceções internas do servidor (falha ao carregar modelo, erro inesperado durante a inferência etc.). Cada categoria deve ser tratada de forma adequada, retornando códigos HTTP coerentes e mensagens úteis.

Lembre-se da convenção HTTP: códigos na faixa 400 indicam erro do cliente (requisição inválida), enquanto 500 indicam erro do servidor (problema interno).

Erros de cliente (Bad Request – 400): se o JSON de entrada estiver ausente ou inválido (por exemplo, sintaxe JSON incorreta), ou se estiver bem formado, mas faltarem campos obrigatórios, retornaremos o código 400 Bad Request.

No Flask, podemos fazer isso simplesmente retornando `jsonify({"error": "descrição"})` junto com o código 400, assim: `return jsonify({"error": "JSON inválido"}), 400`. Similarmente, se detectarmos que um campo essencial não foi fornecido, podemos retornar algo como `{"error": "Campo 'sepal_length' ausente"}` com status

400. Essa prática informa ao cliente que ele fez algo errado na requisição, permitindo correção. É interessante enviar mensagens claras para facilitar a vida de quem integra com sua API. No nosso código de exemplo, isso significará checar `if data is None:` e `if 'tal_campo' not in data:` e retornar esses erros adequados.

Erros internos do servidor (Internal Server Error – 500): apesar de todos os cuidados, podem ocorrer exceções durante o processamento: por exemplo, o modelo não consegue calcular a predição por algum motivo inesperado, ou ocorre um erro de tipo em tempo de execução. Nesses casos, o problema não é culpa do cliente, então o certo é retornar código 500. Podemos usar um `try/except Exception` envolvendo a lógica de predição e, em caso de qualquer exceção não prevista, fazer `return jsonify({"error": "erro interno na predição"}), 500`.

Em ambiente de produção, talvez teríamos um log do erro completo no servidor para depuração, mas não enviar dados ao cliente por segurança. Para propósitos de debugging em desenvolvimento, às vezes se envia o traceback no JSON, mas em produção real é melhor não expor detalhes.

Para implementar isso no Flask, temos duas abordagens: lidar manualmente nas funções de rota (usando condicionais e `try/except` como descrito); ou usar `error handlers globais` do Flask (`@app.errorhandler`). Em APIs simples, a primeira abordagem basta. Já `handlers globais` são úteis para capturar exceções específicas em toda a aplicação. No nosso exemplo, faremos manualmente para clareza.

Hands ON - api_ml.py

Passo 1 - Criar aplicação e carregar o modelo:

```
from flask import Flask, request, jsonify
import pickle
import numpy as np

# Criar aplicação Flask
app = Flask(__name__)

# =====
# CARREGAMENTO DO MODELO
# =====
try:
    with open('modelo_iris.pkl', 'rb') as f:
        modelo = pickle.load(f)
    with open('classes_iris.pkl', 'rb') as f:
        classes = pickle.load(f)
    print("✅ Modelo carregado com sucesso!")
except FileNotFoundError:
    print(" Erro: Arquivos do modelo não encontrados!")
    modelo = None
    classes = None
```

Figura 11 – Criar aplicação Flask
Fonte: Elaborado pelo autor (2026)

Passo 2 - Criando as Rotas:


```
@app.route('/')
def home():
    """Rota principal - informacoes da API"""
    return jsonify({
        'nome': 'API de Classificacao Iris',
        'versao': '1.0',
        'descricao': 'API para classificar especies de flores Iris',
        'endpoints': {
            'GET /': 'Informacoes da API',
            'GET /health': 'Status de saude',
            'POST /predict': 'Fazer previsao'
        },
        'exemplo_entrada': {
            'sepal_length': 5.1,
            'sepal_width': 3.5,
            'petal_length': 1.4,
            'petal_width': 0.2
        }
    })

@app.route('/health')
def health():
    """Verifica se a API está funcionando"""
    status = 'healthy' if modelo is not None else 'unhealthy'
    return jsonify({
        'status': status,
        'modelo_carregado': modelo is not None
    })
```

Figura 12 – Rotas Principais
Fonte: Elaborado pelo autor (2026)

```
@app.route('/predict', methods=['POST'])
def predict():
    """
    Rota de previsão - recebe features e retorna a classe prevista.

    Espera um JSON com:
    - sepal_length: comprimento da sépala (cm)
    - sepal_width: largura da sépala (cm)
    - petal_length: comprimento da pétala (cm)
    - petal_width: largura da pétala (cm)
    """

    # Verificar se o modelo está carregado
    if modelo is None:
        return jsonify({
            'erro': 'Modelo não carregado',
            'mensagem': 'O modelo de ML não foi inicializado corretamente'
        }), 500 # Internal Server Error

    # Tentar obter o JSON da requisição
    try:
        dados = request.get_json()
    except Exception as e:
        return jsonify({
            'erro': 'JSON inválido',
            'mensagem': 'O corpo da requisição deve ser um JSON válido'
        }), 400 # Bad Request

    # Verificar se dados foram recebidos
    if dados is None:
        return jsonify({
            'erro': 'Dados não fornecidos',
            'mensagem': 'Envie um JSON com as features da flor'
        }), 400

    # Validar campos obrigatórios
    campos_obrigatorios = ['sepal_length', 'sepal_width', 'petal_length', 'petal_wi
    campos_faltando = [c for c in campos_obrigatorios if c not in dados]

    if campos_faltando:
        return jsonify({
            'erro': 'Campos obrigatórios faltando',
            'campos_faltando': campos_faltando,
            'campos_esperados': campos_obrigatorios
        }), 400
```

Figura 13 – Rotas Predict-1
Fonte: Elaborado pelo autor (2026)

```
# Extrair e validar os valores
try:
    features = np.array([[
        float(dados['sepal_length']),
        float(dados['sepal_width']),
        float(dados['petal_length']),
        float(dados['petal_width'])
    ]])
except (ValueError, TypeError) as e:
    return jsonify({
        'erro': 'Valores inválidos',
        'mensagem': 'Todos os campos devem ser números válidos'
    }), 400

# Fazer a previsão
try:
    predicao_idx = modelo.predict(features)[0]
    probabilidades = modelo.predict_proba(features)[0]

    # Montar resposta
    resposta = {
        'sucesso': True,
        'predicao': {
            'classe': classes[predicao_idx],
            'classe_idx': int(predicao_idx)
        },
        'probabilidades': {
            classes[i]: round(float(prob), 4)
            for i, prob in enumerate(probabilidades)
        },
        'entrada_recebida': dados
    }

    return jsonify(resposta)

except Exception as e:
    return jsonify({
        'erro': 'Erro na previsão',
        'mensagem': str(e)
    }), 500
```

Figura 14 – Rotas Predict-2
Fonte: Elaborado pelo autor (2026)

Por fim, a execução do app:

```
147 if __name__ == '__main__':  
148     print(" Iniciando API de ML...")  
149     app.run(debug=True, port=8000)  
150
```

Figura 15 – Executar API
Fonte: Elaborado pelo autor (2026)

Agora vamos trabalhar com o POSTMAN para testarmos nossa API! Depois de criarmos uma conta e instalarmos o Postman Desktop Agent, podemos utilizar ele para testarmos as rotas da nossa API local. Vamos primeiro testar a saúde da API.

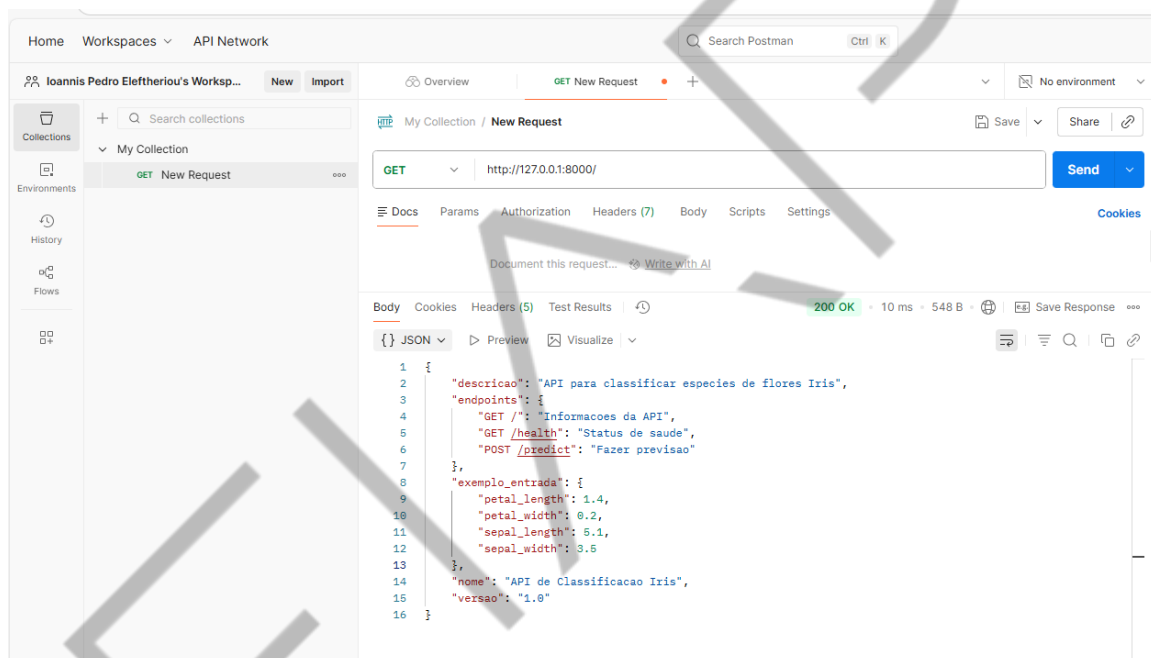


Figura 16 – Testando GET Postman
Fonte: Elaborado pelo autor (2026)

Agora, vamos criar uma nova requisição POST Predict. Na aba “Body”, vamos selecionar “raw” e “JSON”, colocar nosso JSON com as FEATURES e tentar prever o tipo de flor.

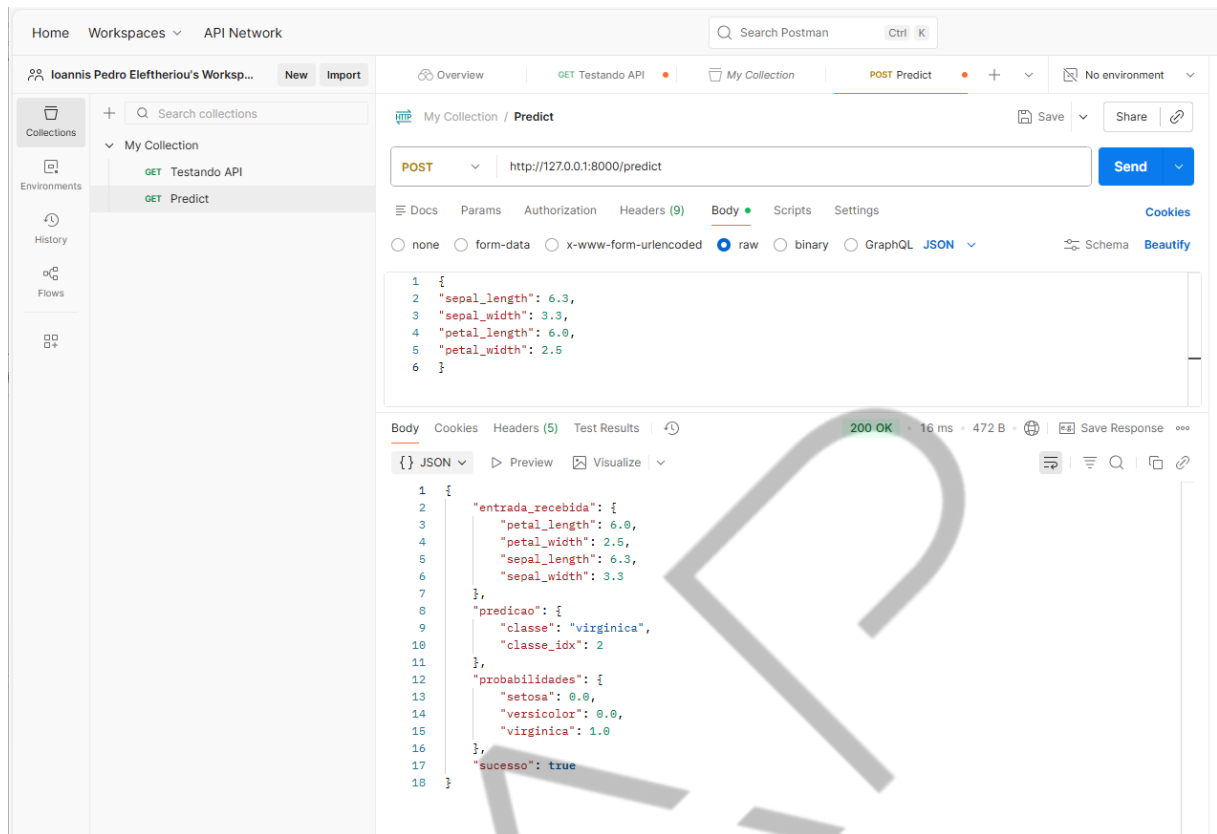


Figura 17 – Testando POST - PREDICT

Fonte: Elaborado pelo autor (2026)

Perfeito! A nossa API foi servida dos dados da flor e nos retornou a previsão identificando que era a flor do tipo *virgínica*! Durante os testes, fique de olho no console onde o Flask está rodando, ele loga cada requisição (método, endpoint, código de status e tempo). Isso ajuda a depurar se a rota está sendo atingida e o resultado obtido.

FLASK em Produção

Agora que temos uma API integrada a um modelo, é importante trazermos algumas considerações sobre o ambiente de produção. Esse app Flask construído da forma que foi feita não é o ideal para ser exposto na internet. Quando executamos ``app.run()``, o Flask usa o servidor Werkzeug, que é single-threaded por natureza e feito apenas para desenvolvimento, não sendo capaz de lidar com alta carga ou

múltiplas conexões simultâneas. Em produção precisamos de uma solução mais robusta.

As práticas recomendadas são utilizar um servidor WSGI (Web Server Gateway Interface), como Gunicorn (Linux) ou Waitress (Windows), para servir a aplicação Flask.

Servidores WSGI, como o Gunicorn, utilizam workers paralelos, gerenciando melhor tráfego alto e requisições concorrentes, ao contrário do servidor interno do Flask, que não é para produção.

Arquitetura Padrão (Nginx + Gunicorn + Flask): coloque um reverse proxy (ex: Nginx ou Apache) na frente do Gunicorn. O Nginx lida com requisições HTTP (portas 80/443), encaminha para o Gunicorn (porta interna 5000) e gerencia arquivos estáticos, SSL/TLS e load balance. Essa arquitetura é padrão e garante resiliência e escalabilidade.

Considerações sobre performance e sincronismo: o Flask utiliza WSGI síncrono: um worker é bloqueado por requisição. Se o modelo de Machine Learning for lento, isso pode limitar a taxa de transferência de requisições.

Soluções de performance: escalonamento horizontal (aumentar o número de workers Gunicorn); migrar para frameworks assíncronos como FastAPI, pois o Flask não suporta nativamente assincronismo.

Para a maioria das aplicações de ML de pequena/média escala, o escalonamento via workers Gunicorn é geralmente suficiente.

Para as próximas aulas, vamos refatorar e aprimorar nossa API, utilizando justamente o FastAPI, um framework moderno e com mais recursos que vão nos ajudar.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, o foco foi a transição do modelo de Machine Learning treinado, que estava em um notebook, para um ambiente operacional via API RESTful utilizando o microframework Flask. Vimos a importância de expor modelos como serviços web para que possam ser integrados a aplicações reais.

Aprendemos a anatomia de uma aplicação Flask básica, desde a sua instanciação e a definição de rotas com decoradores, até a utilização do formato JSON para entrada e saída de dados, o que é fundamental para a comunicação entre sistemas.

Não se esqueçam de assistir às videoaulas e também se aprofundar na documentação do FLASK para continuar praticando e evoluindo sua API.

REFERÊNCIAS

FLASK-PTBR.READTHEDOCS.IO. **Documentação do Flask (em português)**. [s.d.] Disponível em: <https://flask-ptbr.readthedocs.io/en/latest/>. Acesso em: 14 jan. 2026.

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps: Operationalizing Machine Learning Models**. Sebastopol: O'Reilly Media, 2021.

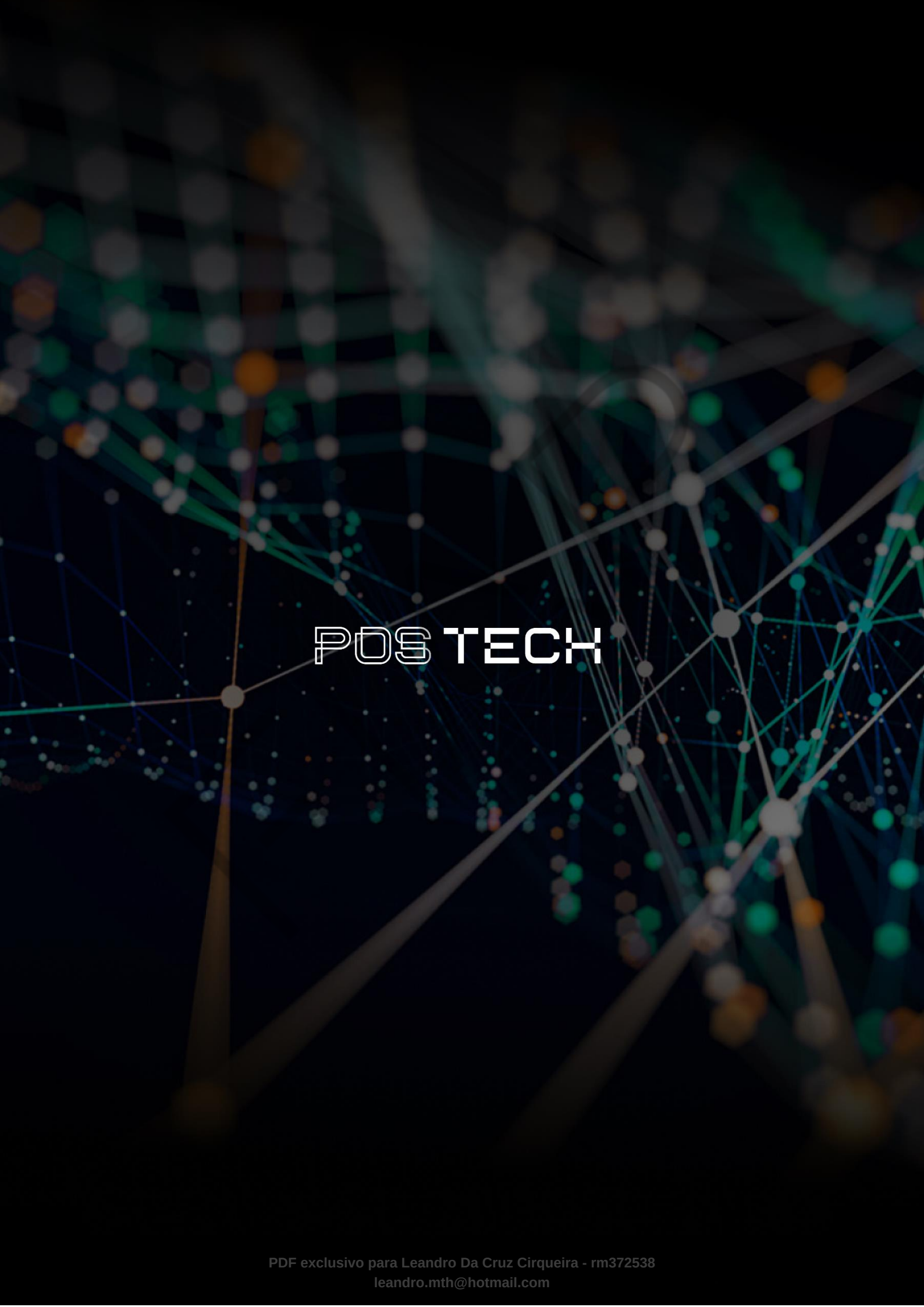
LUBANOVIC, B. **FastAPI: Modern Python Web Development**. Sebastopol: O'Reilly Media, 2023.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2021.

PALAVRAS-CHAVE

FLASK. API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON. MLOps.

EMSE



POSTECH